Reviewer Pack — Minimal Logarithmic Growth of Algebraic K-Theory Rank and Co…

Entry d20580b5b94b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 04:26:46 UTC

Minimal Logarithmic Growth of Algebraic K-Theory Rank and Communication Complexity Rank Inequality

Entry ID: `d20580b5b94b` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-05 04:26:46 UTC

1. Conjecture

Field A (mathematical branch): Algebra (Algebraic K-theory) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

> For all Boolean functions f with n input bits, the minimal logarithmic growth rate of the rank of its associated algebraic K-group is linearly correlated with its communication complexity rank, such that log(rank(K(f))) = Θ(communication_rank(f)) and communication_rank(f) ≥ O(log(rank(K(f)))).

Rationale (proposer's reasoning):

> Algebraic K-theory provides a robust framework to study the structure of rings, which can be applied to Boolean functions through their associated algebraic structures. The rank of the algebraic K-group captures the complexity of these structures, potentially reflecting communication complexity. A strong correlation between these two measures could indicate a deep connection between ring theory and complexity theory.

Taxonomy category: `communication_complexity_rank` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `fc377a7aa4deb7ee`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For all Boolean functions f with n input bits, we consider a result supported if the correlation coefficient from linear regression of log(rank(K(f))) against communication_rank(f) is ≥ 0.8 for all tested n ≤ 40 and the slope is within ±3 standard deviations of the expected ratio Θ(communication_rank(f)). A result is falsified if any seed produces a correlation coefficient < 0.5 or a slope outside the ±3 standard deviation range.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - algebraic k-theory AND communication complexity rank - matrix rank IN communication complexity AND algebraic K-group - logarithmic growth rate rank (algebraic K-theory) related to communication complexity rank

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
from math import log, sqrt

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def communication_complexity(f):
        n = len(f)
        max_communication = 0
        for i in range(2**n):
            comm = sum(f[j] != f[i ^ j] for j in range(n))
            if comm > max_communication:
                max_communication = comm
        return max_communication

    def compute_k_group_rank(f):
        n = len(f)
        k_group_rank = 0
        for i in range(2**n):
            subset_sum = sum(f[j] for j in range(n) if (i >> j) & 1)
            if subset_sum == 0:
                k_group_rank += 1
        return k_group_rank

    def linear_regression(x, y):
        n = len(x)
        mean_x = sum(x) / n
```

```
        mean_y = sum(y) / n
        numerator = sum((x[i] - mean_x) * (y[i] - mean_y) for i in range(n))
        denominator = sqrt(sum((x[i] - mean_x)**2 for i in range(n)) * sum((y[i] - mean_y)**2 for
        if denominator == 0:
            return None, None
        slope = numerator / denominator
        intercept = mean_y - slope * mean_x
        return slope, intercept

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        f = generate_boolean_function(n)
        communication_rank = communication_complexity(f)
        k_group_rank = compute_k_group_rank(f)
        log_k_group_rank = log(k_group_rank) if k_group_rank > 0 else None
        results.append((n, communication_rank, log_k_group_rank))

    x = [r[1] for r in results]
    y = [r[2] for r in results if r[2] is not None]

    if len(y) < 2:
        return {
            "metric_name": "correlation_coefficient",
            "metric_value": None,
            "instances_tested": len(results),
            "n_max": max(n_values),
            "conjecture_holds": False,
            "counterexample": "not_enough_valid_data"
        }

    slope, intercept = linear_regression(x, y)
    if slope is None:
        return {
            "metric_name": "correlation_coefficient",
            "metric_value": None,
            "instances_tested": len(results),
            "n_max": max(n_values),
            "conjecture_holds": False,
            "counterexample": "division_by_zero"
        }

    correlation_coefficient = sum((y[i] - (slope * x[i] + intercept))**2 for i in range(len(y))) /
    return {
        "metric_name": "correlation_coefficient",
        "metric_value": correlation_coefficient,
        "instances_tested": len(results),
        "n_max": max(n_values),
        "conjecture_holds": correlation_coefficient >= 0.8 and abs(slope) <= 3 * sqrt(correlation_
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2**i + 1 for i in range(5, 8)]
```

```
    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_metric_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) /
    std_metric_value = sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results if r['
    support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    elif any(not r["conjecture_holds"] and r["metric_value"] < 0.5 for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"correlation_coefficient_too_low\" first_failing
    elif any(not r["conjecture_holds"] and abs(r["metric_value"] - 0) > 3 * sqrt(r["metric_value"]
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"slope_outside_range\" first_failing_seed={first
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_865f62da.py", line 104, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_865f62da.py", line 59, in run_trial
    communication_rank = communication_complexity(f)
                         ^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_865f62da.py", line 28, in communication
    comm = sum(f[j] != f[i ^ j] for j in range(n))
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_865f62da.py", line 28, in <genexpr>
    comm = sum(f[j] != f[i ^ j] for j in range(n))
                       ~^~~~~~~
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed before producing data, which means it did not complete its execution to provide a correlation coefficient and slope for the analysis. | next: Re-run the test with proper error handling to ensure that it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12952
2	propose	ollama_remote	glm4:latest	0	0	17686
3	preregistration	ollama_remote	glm4:latest	0	0	15256
4	novelty	ollama_remote	glm4:latest	0	0	8495
5	novelty	ollama_remote	glm4:latest	0	0	9580
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16187
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12161
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10919
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13457
10	judge	ollama_remote	glm4:latest	0	0	11986

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 128679 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/d20580b5b94b.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/d20580b5b94b.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/d20580b5b94b.tar.gz` (if generated)